

TP ROBOT : Parcours d'un labyrinthe

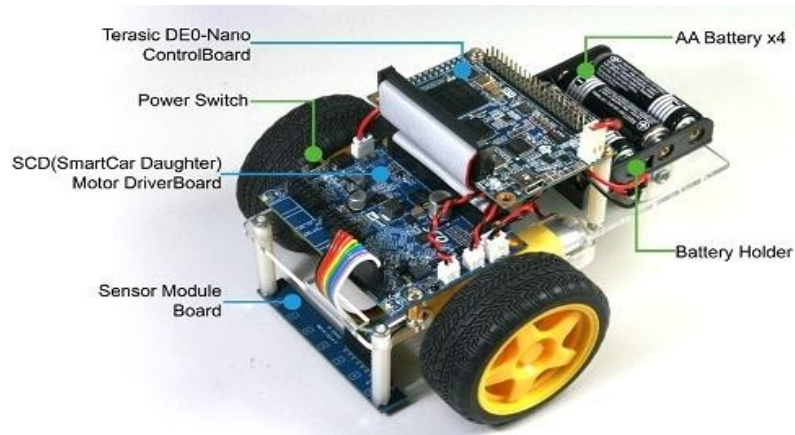
ET5 EIE – Module SOC

Questions/Préparation

- 1- Le concept SoC est-il applicable à la DE0-NANO ? Justifier votre réponse.
- 2- Vous allez utiliser la DE0-NANO en séance de TP.
 - a. Quelles sont ses différences par rapport à la DE0-NANO-SoC ? Dans ce dernier cas, le qualificatif SoC est-il justifié ?
 - b. Identifier les hardcore et softcore utilisables pour chaque circuit FPGA de chaque carte
- 3- Dans le cas de ces deux cartes :
 - a. Précisez les systèmes d'interconnexions utilisés.
 - b. Les comparer
 - c. Dédurre :
 - i. s'ils peuvent constituer une hiérarchie d'interconnexion
 - ii. comment on peut faire communiquer deux éléments rattachés aux systèmes d'interconnexion identifiés
- 4- Le processeur NIOS II est-il un soft IP core, un firm IP core ou un hard IP core ? Justifier votre réponse.
- 5- Pour un processeur donné, l'unité arithmétique et logique est-elle extensible ? Précisez dans le cas particulier du soft processeur NIOS II ?
- 6- Donner le flot de développement utilisable pour la DE0-NANO ? Identifier le Codesign
- 7- Plateforme robot ALTERA A-CUTE CAR
 - a. Donner son schéma synoptique
 - b. Capteurs et actionneurs. Précisez :
 - I. Mode(s) de fonctionnement
 - II. leurs interfaçages au NIOS II
 - III. Comment lire une valeur du capteur ? Envoyer une valeur à un actionneur ?
 - c. Est-ce que le traitement peut être porté au plus près du capteur ? de l'actionneur ? Préciser :
 - I. Pourquoi ?
 - II. Comment choisir ?
 - III. Comment faire ?Sachant le robot utilisé, en donner deux fonctions possibles (une pour le capteur et l'autre pour l'actionneur)

Travail à faire

On souhaite concevoir un système de contrôle commande d'un robot suiveur de ligne basé sur une carte FPGA. L'objectif est de permettre au robot de trouver la sortie d'un labyrinthe. La plateforme robot est celle de ALTERA A-CUTE CAR. Vous trouverez toutes les informations relatives à ce robot sur le site de Terasic.



Pour pouvoir parcourir le labyrinthe, des IPs ont été développés par des sous-traitants : calcul de la position de la ligne, automate de suivi de ligne et automate de rotation.

Dans la solution finale visée, ces IPs seront associés à un processeur embarqué qui sera chargé de les contrôler pour permettre le parcours du labyrinthe puis de simplifier le parcours pour trouver, dans une deuxième passe, directement la sortie sans emprunter d'impasse.

Le travail à réaliser se déroulera en plusieurs étapes qui constituent les livrables successifs à fournir :

1. **Construire à l'aide de Qsys** (ou Platform Designer) le Système sur puce que vous utiliserez avec les IP fournis pour :
 - faire effectuer au robot un suivi de ligne
 - lui faire faire demi-tour lorsqu'il n'y a plus de ligne, pour la retrouver et la suivre à nouveau. La direction dans laquelle est faite le demi-tour devra changer à chaque nouveau demi-tour

Pour construire le SOC, prévoir :

- Un **générateur d'horloge** externe
- Un **contrôleur pour la mémoire SDRAM** (Mots de 16 bits, 1 Chip Select, 4 Banques, 13 rangées et 9 colonnes, garder les valeurs de timing par défaut)
- Un **processeur NIOS**
- Un **UART JTAG** pour permettre l'envoi du programme exécuté par le processeur et le débogage.
- Un identifiant nommé **sysid** qui permet de numéroté le processeur généré pour vérifier que l'architecture implantée sur le FPGA est bien la dernière qui a été générée par Qsys.
- Un **timer**
- **Les ports d'Entrée/Sortie nécessaires.**

Une fois le système généré par Qsys, le rajouter dans le fichier VHDL principal du projet Quartus et implanter le système sur le FPGA du Robot.

Exécuter ensuite le programme C fourni pour que le robot se comporte comme souhaité.

2. **Modifier l'automate suivi de ligne pour permettre la détection des intersections** : créer un nouvel état « intersection » dans lequel le robot se déplace en ligne droite et dans lequel l'automate renvoie la valeur des capteurs seuillés simplifiés sur 3 bits pour permettre au NIOS, en analysant leurs valeurs successives, de déterminer la nature des intersections traversées par le Robot. Ecrire et valider le programme C correspondant. Une fois une intersection traversée et reconnue, le NIOS commande au robot de s'arrêter et le programme doit afficher les commandes à envoyer aux automates en fonction de l'intersection détectée (Cf question 5 - sans le faire réellement) en utilisant une fonction écrite en C.
3. **Créer une instruction personnalisée pour le NIOS** permettant de fournir directement la commande des automates en fonction du type d'intersection détecté. Après avoir validé le modèle VHDL en simulation et vérifié sa fréquence max de fonctionnement, l'associer au NIOS et l'intégrer dans le programme C. Comparer les performances par rapport à l'utilisation de la fonction C précédente.
4. **Modifier l'automate de suivi de ligne** pour qu'il effectue en plus, la détection des intersections et l'envoie au NIOS le type d'intersection traversée. Il doit aussi être capable de reconnaître l'arrivée. Là encore, le robot doit s'arrêter après chaque intersection traversée et le programme se contente d'afficher les commandes à envoyer aux automates, calculées par l'instruction personnalisée.
5. En utilisant le SOC précédent, **écrire le programme C permettant d'explorer le labyrinthe pour en trouver la sortie**. Cette dernière est repérée par une large zone noire (beaucoup plus large que la ligne suivie par le robot). Le principe est, à chaque intersection rencontrée, de tourner si possible à droite et de reprendre le suivi de ligne. S'il ne peut tourner à droite, le robot va tout droit et si c'est impossible, il tourne à gauche. S'il arrive à une impasse, il fait demi-tour. S'il n'y a pas de boucle dans le labyrinthe (ce que nous supposons), il trouvera ainsi, au bout d'un moment, la sortie. Le programme doit mémoriser la séquence de décisions prises à chaque intersection rencontrée jusqu'à ce que la sortie soit trouvée. Il doit afficher cette séquence quand celle-ci est atteinte.
6. **Ecrire une fonction C qui simplifie la séquence de décisions** pour éliminer toutes les impasses du parcours puis utiliser la séquence simplifiée pour que le robot parcoure désormais le labyrinthe sans emprunter les impasses.